Preliminary Results on Evaluating LLMs for Labeling Root Cause Categories of Fixed Flaky Tests

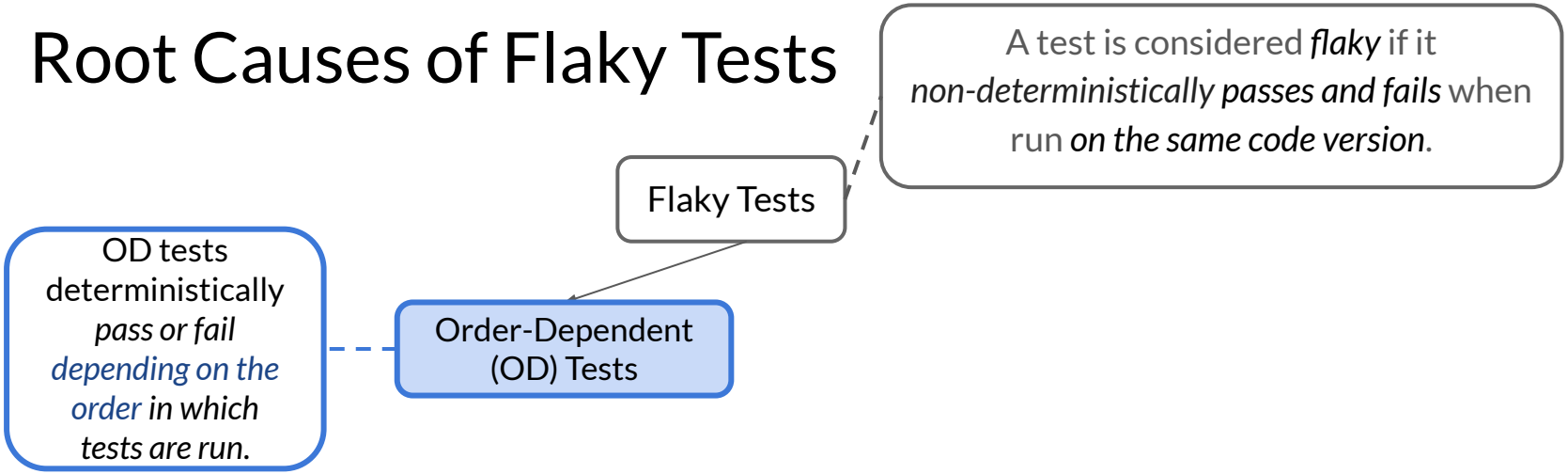
Yang Chen, Kaiyao Ke, Darko Marinov



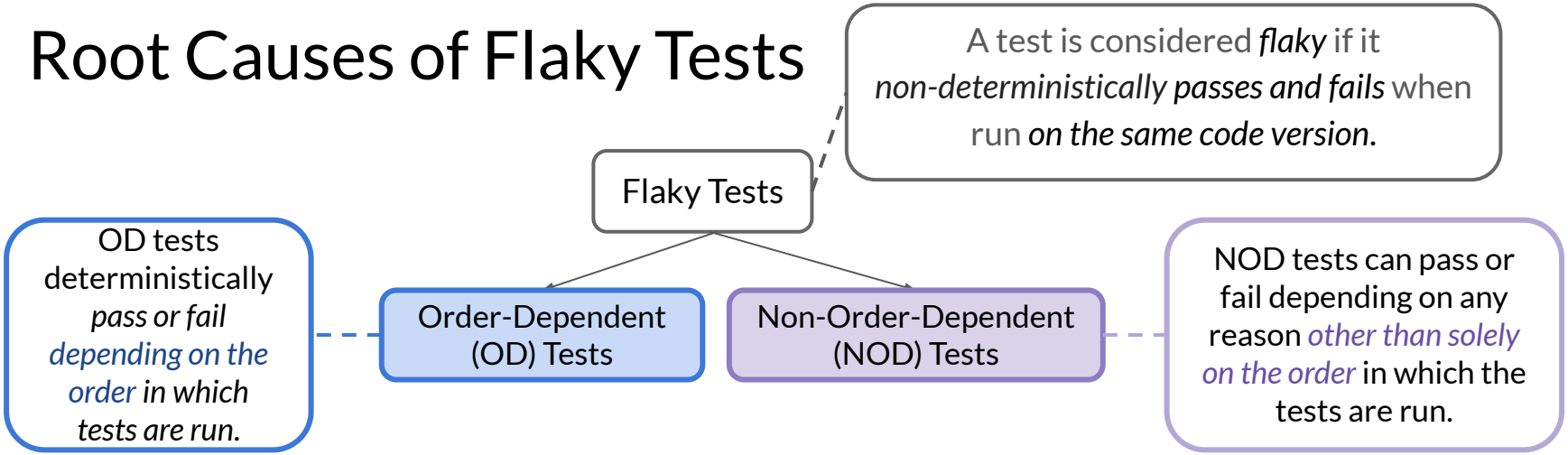


Flaky Test Workshop 2026

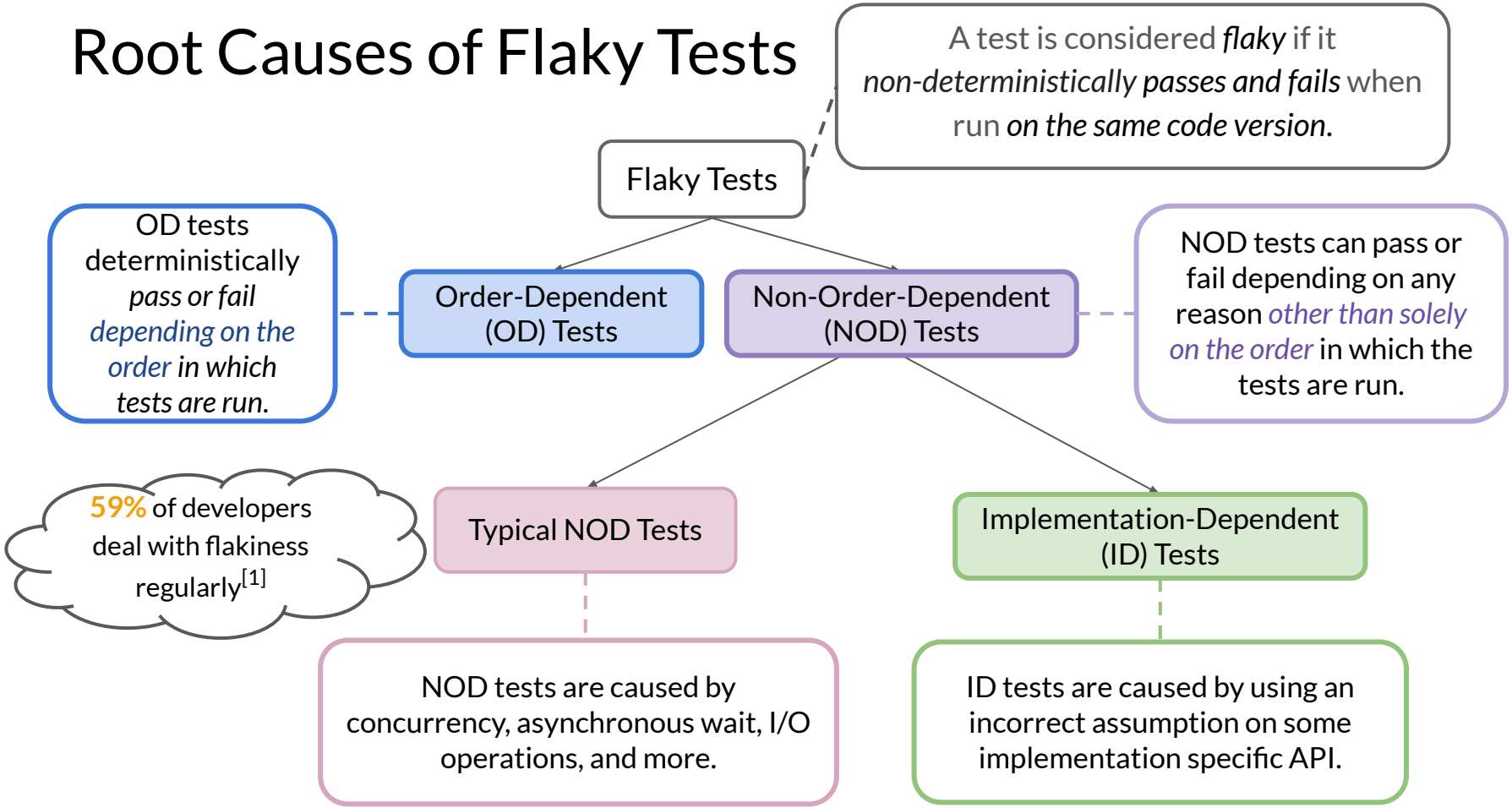
Root Causes of Flaky Tests



Root Causes of Flaky Tests



Root Causes of Flaky Tests



[1] A Survey of Flaky Tests. Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. TOSEM 2021.

Existing Work on Labeling Flaky Tests

Prior work on labeling flaky tests has mostly focused on determining ***whether*** a test is flaky or not. ***Much less attention has been devoted to determining the category of flakiness (the root cause).***

Knowing the flakiness category helps developers decide whether to rerun a failure, prioritize which tests to fix, or assign the fix to the right developer.

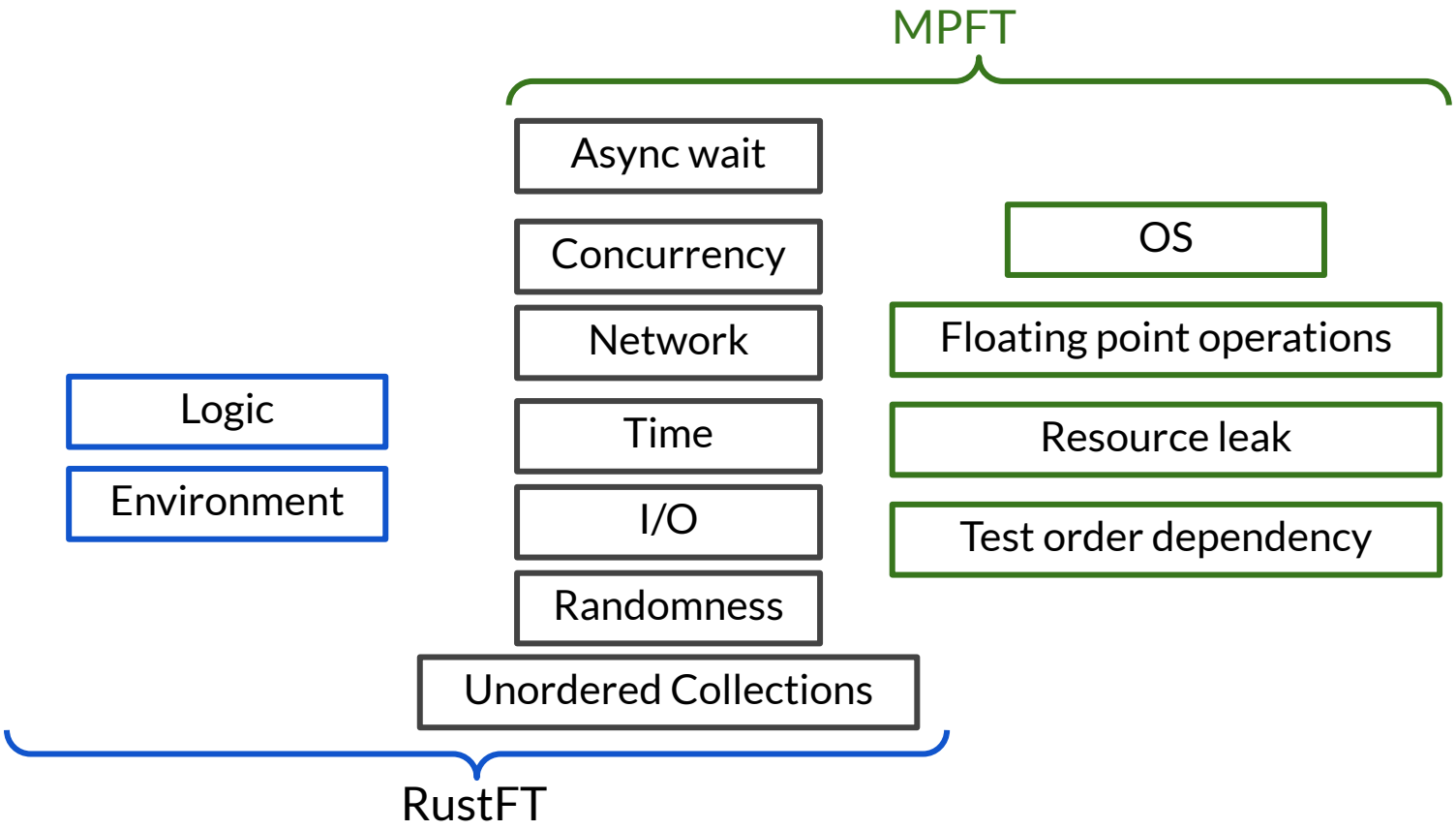
Labeling ***unfixed tests*** from failure alone (stack trace, test code, code under test) is ***challenging***.

In this paper, we focus on ***labeling already fixed tests***, helps studying historical fixes to advance tools for detecting, mitigating, and fixing flaky tests.

Study – Dataset

Dataset	**RustFT** *(Schroeder et al., 2025)*	**MPFT** *(new dataset from course MP)*
Tests	52 fixed flaky tests from Github Issues/Pull Requests	244 fixed flaky tests from Github Issues/Pull Requests
Language	Rust	17 programming languages
Categories	9 categories	11 categories
Status	Public on GitHub, may be in LLM training data	New, not in LLM training data
Format	Issue description + Flaky test patch	

Study – Root Cause Catogeries



Study – Labeling Process of MPFT

- Ground truth labels:
  - 36 students involved for labeling
  - Each test gets exactly ***one*** (the dominant) root cause category
  - Each test labeled independently by ***two or more*** researchers/students to ensure correctness
  - In case of conflict: label from the most experienced researcher wins

Study – Prompt Template

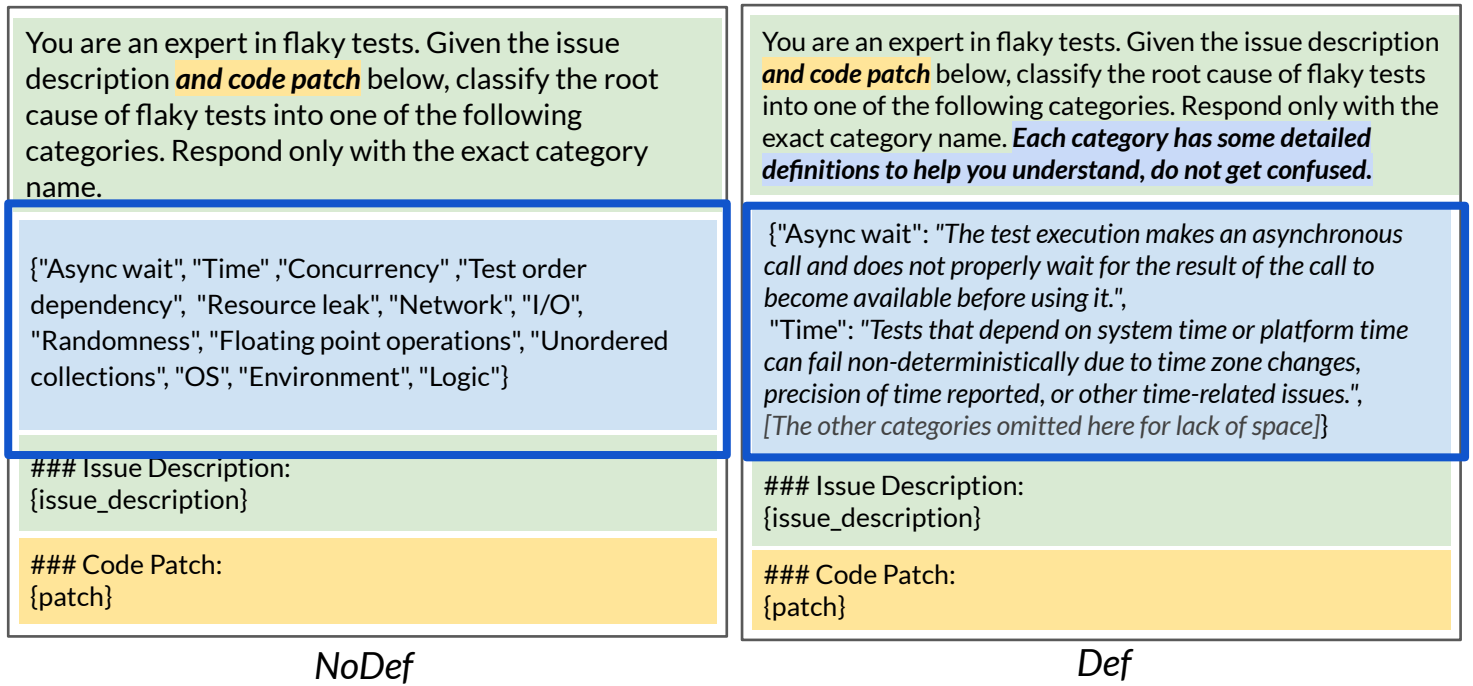
NoDef + NoPatch
Only category names listed. No definitions and code diff.
Tests how well models classify from issue description alone.

Def + NoPatch
Category names and definitions. No code patch. Tests whether more clarity improves classification.

NoDef + Patch
Category names and full code diff (PR patch). Simulates knowing the fix but lacking explicit category guidance.

Def + Patch
Definitions and code patch are both provided. Best-case setting for LLM classification performance.

Study – Prompt Template



Evaluation

- Two datasets: RustFT, MPFT
- Models: GPT-4 and GPT-5
- Four prompt templates
- Research Questions:
  - How effectively can LLMs label root cause?
  - Why LLMs fail to identify the root causes of flakiness?

Evaluation – Success Rate

Table 1: Correct model predictions across all settings

Dataset	RustFT (Total #Tests: 52)				MPFT (Total #Tests: 244)			
Model	GPT-4		GPT-5		GPT-4		GPT-5	
Prompt	NoDef	Def	NoDef	Def	NoDef	Def	NoDef	Def
NoPatch	25	25	25	26	94	99	122	136
Patch	25	25	37	38	113	120	142	151
Avg.	**25.0**	**25.0**	**31.0**	**32.0**	**103.5**	**109.5**	**132**	**143.5**

- LLMs achieve accuracies ranging from 38% (94/244) to 61% (151/244), which is not very high, but note that it is still much better than random guess because we do not consider only binary classification (whether a test is flaky or not)
- Overall accuracy is similar on both datasets

Evaluation — Comparison between Models

Table 1: Correct model predictions across all settings

Dataset	RustFT (Total #Tests: 52)				MPFT (Total #Tests: 244)			
Model	GPT-4		GPT-5		GPT-4		GPT-5	
Prompt	NoDef	Def	NoDef	Def	NoDef	Def	NoDef	Def
NoPatch	25	25	25	26	94	99	122	136
Patch	25	25	37	38	113	120	142	151
Avg.	**25.0**	**25.0**	**31.0**	**32.0**	**103.5**	**109.5**	**132**	**143.5**

- GPT-5 consistently outperforms GPT-4 in every configuration
- The performance gap between the two models is smaller on the RustFT dataset than on MPFT
- Interestingly, GPT-4 demonstrates a consistent precision rate on RustFT (25 out of 52 tests), although the exact correctly predicted instances can differ

Evaluation — Comparison between Prompts

Table 1: Correct model predictions across all settings

Dataset	RustFT (Total #Tests: 52)				MPFT (Total #Tests: 244)			
Model	GPT-4		GPT-5		GPT-4		GPT-5	
Prompt	NoDef	Def	NoDef	Def	NoDef	Def	NoDef	Def
NoPatch	25	25	25	26	94	99	122	136
Patch	25	25	37	38	113	120	142	151
Avg.	**25.0**	**25.0**	**31.0**	**32.0**	**103.5**	**109.5**	**132**	**143.5**

- Comparing between ***NoDef*** and ***Def***, we observe that adding detailed definitions of root causes helps models understand the context and reduces potential ambiguity in inference, leading to an average relative improvement ranging from 0% to 8.7%.

Evaluation — Comparison between Prompts

Table 1: Correct model predictions across all settings

Dataset	RustFT (Total #Tests: 52)				MPFT (Total #Tests: 244)			
Model	GPT-4		GPT-5		GPT-4		GPT-5	
Prompt	NoDef	Def	NoDef	Def	NoDef	Def	NoDef	Def
NoPatch	25	25	25	26	94	99	122	136
Patch	25	25	37	38	113	120	142	151
Avg.	**25.0**	**25.0**	**31.0**	**32.0**	**103.5**	**109.5**	**132**	**143.5**

- Comparing between ***NoPatch*** and ***Patch***, we find that including code changes helps improve model performance with an average increase of 0% to 21.2%.

Evaluation — Failure Analysis

- 107 misprediction instances under GPT-5 *Patch + Def* setting

Reason 1 (#N=82): Misunderstanding the root cause.
Example: when a patch increases timeout values to mitigate a flaky test caused by async wait, the model ***focuses only on the timing aspect*** and incorrectly categorizes the root cause as Time instead of Async wait.

Evaluation — Failure Analysis

- 107 misprediction instances under GPT-5 *Patch + Def* setting

Reason 1 (#N=82): Misunderstanding the root cause.

Reason 2 (#N=18): Misunderstanding the definition of categories

Example: Concurrency is defined as nondeterminism from thread interactions (excluding async). ***LLMs often confuse Async Wait with Concurrency***, likely focusing on the multithreaded aspect rather than the async synchronization issue.

Evaluation — Failure Analysis

- 107 misprediction instances under GPT-5 *Patch + Def* setting

Reason 1 (#N=82): Misunderstanding the root cause.

Reason 2 (#N=18): Misunderstanding the definition of categories

Reason 3 (#N=7): Insufficient code context

Example: ***Unordered collections defined in external libraries*** that are not in the modified files, so models lack sufficient context to understand the underlying cause of flakiness, leading to mispredictions.

Evaluation — Failure Analysis

- 107 misprediction instances under GPT-5 *Patch + Def* setting

Reason 1 (#N=82): Misunderstanding the root cause.

Reason 2 (#N=18): Misunderstanding the definition of categories

Reason 3 (#N=7): Insufficient code context

Conclusion

- A preliminary evaluation of LLMs for labeling the root causes of fixed flaky tests
- LLMs achieve accuracies ranging from 38% to 61%
  - Definitions and code patches together give the biggest accuracy boost with up to 21% over the baseline no-definition, no-patch prompt
- Failure analysis of three root causes:
  - LLM misunderstanding the root cause (82/107)
  - LLM misunderstanding the category definition (18/107)
  - Insufficient code context (7/107)

 yangc9@illinois.edu

 flakyrootcause-create/FlakyRootCause